

Emotion Classification in Context Texts using Traditional Machine Learning and BERT

Zhang Yibing 123090837
School of Data Science
Chinese University of Hong Kong, Shenzhen
yibingzhang@link.cuhk.edu.cn

December 3, 2025

1 Introduction and Related Work

Emotion recognition in text is a critical NLP task [1], classifying text into categories such as **joy, sadness, anger, fear, and neutral** [2]. This field has seen advancements from traditional machine learning methods to modern contextual models [3]. Analyzing the emotional cues implied in the text also greatly promotes the development of the language model [4]. This project compares two approaches using a **publicly available online corpus**: 1) Traditional Machine Learning (ML) with TF-IDF and classical classifiers (NB, LR, RF, SVM), and 2) a state-of-the-art Deep Learning approach using the **BERT** transformer model. This project explores the critical correlation between feature-engineered ML models and highly contextual, pre-trained transformers and their capacity for effective context comprehension and linguistic sentiment prediction. The study is anchored by detailed performance comparison, aims to identify the most efficient approach for the specific predictive task.

2 Approach

2.1 Traditional Machine Learning Pipeline

Text preprocessing used **spaCy** for cleaning, lemmatization, and stop word removal. Features were extracted using **TfidfVectorizer** with **Unigrams and Bigrams** (`ngram_range = (1, 2)`). The vectorizer was fitted on the entire corpus (7,934 texts). Four classifiers (**Multinomial Naive Bayes (NB)**, **Logistic Regression (LR)**, **Random Forest (RF)**, and **Linear Support Vector Classifier (SVC)**) were trained and evaluated on the test set.

2.2 Deep Learning Pipeline (BERT)

The BERT model was initialized via the `ktrain` library. The input text was tokenized and formatted with a maximum length of 350. The model was fine-tuned using the **one-cycle learning rate schedule** for **3 epochs** with a max learning rate of $2e - 5$. This approach leverages BERT’s contextual embeddings, eliminating manual feature engineering.

3 Experiments

3.1 Data and Evaluation Method

The dataset was sourced from a publicly available corpus, split into a Training Set (7,934 texts) and a Test Set (3,393 texts). The five emotion classes are primarily balanced (e.g., 'joy' is $\approx 2,326$, 'fear' is $\approx 2,171$). Performance was primarily measured using the **micro-averaged F1 Score** and **Accuracy** on the test set.

3.2 Experiment Results and Analysis

3.2.1 Performance Comparison

The performance metrics for both traditional ML and BERT are summarized in Table 1.

The **Logistic Regression** achieved the highest score among traditional models, demonstrating robustness in the high-dimensional TF-IDF space. However, the **BERT** model significantly outperformed all traditional methods, achieving a 82.42% F1-Score. This 14.01% **point gain** confirms the superior ability of transformer models to capture complex emotional context over simple bag-of-words (TF-IDF) representations.

3.2.2 Confusion Matrix Analysis

The normalized confusion matrices (Figure 1) illustrate class-specific performance. Some traditional models indicated difficulties when distinguishing between **sadness** and **neutral**, and between **joy** and **neutral**.

4 Conclusion

This comparative study demonstrated a significant performance gap between traditional methods and pre-trained contextual models in emotion classification, which validates the need for contextual models in NLP tasks requiring nuanced semantic understanding. A concrete example illustrating BERT’s superior capture of subtle textual cues, which LR failed to correctly interpret, are provided in the appended computational results starting from **page 18 of the supplementary material**. Both model pipelines have been saved for future deployment.

References

A Appendix

Table 1: Summary of Classification Results

Approach	Accuracy (%)	F1-Score (%)
Naive Bayes	62.81	62.81
Random Forest	65.69	65.69
Logistic Regression	68.41	68.41
SVM (Linear)	67.40	67.40
BERT	82.42	82.42

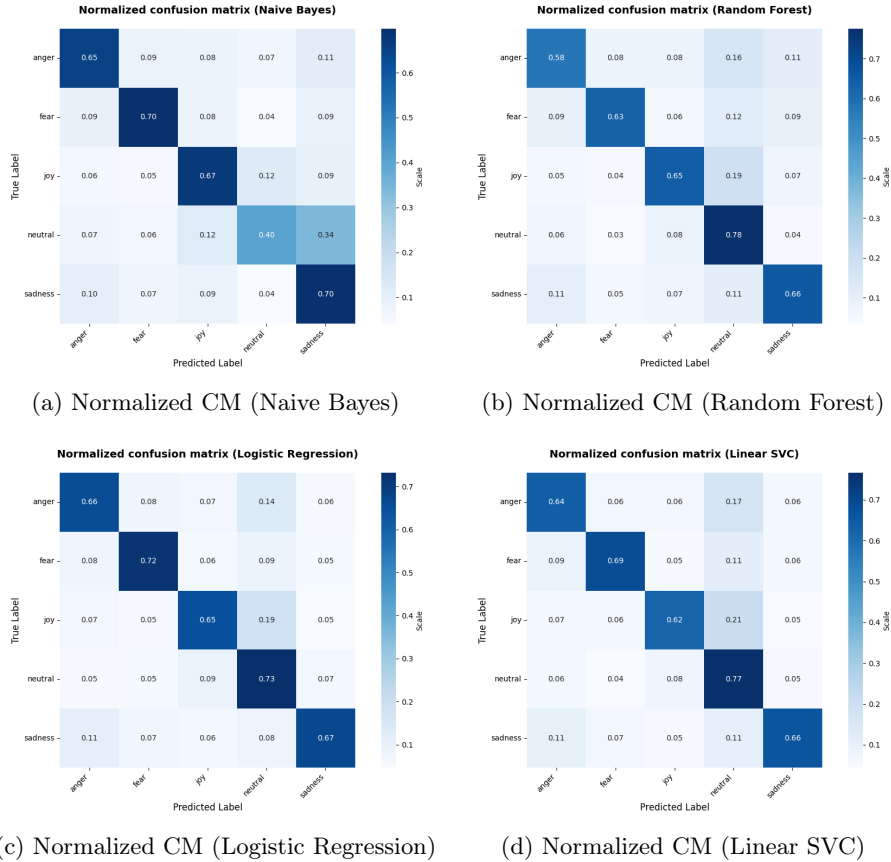


Figure 1: Normalized Confusion Matrices for Traditional Models.

Emotion Classification in Texts using Traditional Machine Learning

This implementation classifies text into five emotion categories (joy, sadness, anger, fear, neutral) using TF-IDF vectorization and multiple classifiers including:

- Naive Bayes
- Logistic Regression
- Random Forest
- SVM.

The pipeline covers text preprocessing, feature extraction, model training, and evaluation using F1 scores and confusion matrices.

```
In [9]: # Importing preparation

# mathematics
import pandas as pd
import numpy as np

# text preprocessing
import re

# plots and metrics
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
from typing import List, Optional

# feature extraction / vectorization
from sklearn.feature_extraction.text import TfidfVectorizer

# classifiers
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline

# save and load a file
import pickle

import warnings
warnings.filterwarnings('ignore')
```

Import Datasets

Text-Emotion Dataset was split into training 70% and testing 30%

```
In [10]: # The two sets both consist of Emotion Column and Text Column.
```

```

df_train = pd.read_csv('data/data_train.csv')
df_test = pd.read_csv('data/data_test.csv')

X_train = df_train.Text
X_test = df_test.Text

y_train = df_train.Emotion
y_test = df_test.Emotion

class_names = ['joy', 'sadness', 'anger', 'neutral', 'fear']
data = pd.concat([df_train, df_test])

print('size of training set: %s' % (len(df_train['Text'])))
print('size of validation set: %s' % (len(df_test['Text'])))
print(data.Emotion.value_counts())

data.head()

```

```

size of training set: 7934
size of validation set: 3393
Emotion
joy      2326
sadness  2317
anger    2259
neutral  2254
fear     2171
Name: count, dtype: int64

```

Out[10]:

	Emotion	Text
0	neutral	There are tons of other paintings that I thin...
1	sadness	Yet the dog had grown old and less capable , a...
2	fear	When I get into the tube or the train without ...
3	fear	This last may be a source of considerable disq...
4	anger	She disliked the intimacy he showed towards so...

```

In [11]: def plot_confusion_matrix(y_true: np.ndarray, y_pred: np.ndarray,
                                   classes: List[str], normalize: bool = False,
                                   title: Optional[str] = None,
                                   cmap: plt.cm = plt.cm.Blues,
                                   figsize: tuple = (10, 7)) -> plt.Axes:

    # 1. Compute the confusion matrix
    cm = confusion_matrix(y_true, y_pred)

    # 2. Apply normalization if requested
    if normalize:
        # Avoid division by zero by adding a small constant to the denominator
        cm = cm.astype('float') / (cm.sum(axis=1)[:, np.newaxis] + 1e-8)
        default_title = "Normalized Confusion Matrix"
        value_format = '.2f'
    else:
        default_title = "Confusion Matrix"
        value_format = 'd'

    # Use provided title or the default one
    plot_title = title if title is not None else default_title

```

```

# 3. Create the visualization
plt.figure(figsize=figsize)

# Use Seaborn's heatmap for a cleaner and more styled plot
ax = sns.heatmap(cm,
                  annot=True,           # Annotate cells with the value
                  fmt=value_format,     # String formatting of annotations
                  cmap=cmap,
                  xticklabels=classes,
                  yticklabels=classes,
                  cbar_kws={'label': 'Scale'},
                  square=True)         # Try to make cells square

# 4. Set Labels and title
ax.set_title(plot_title, fontsize=14, fontweight='bold', pad=20)
ax.set_xlabel('Predicted Label', fontsize=12)
ax.set_ylabel('True Label', fontsize=12)

# 5. Improve tick labels for better readability
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')
ax.set_yticklabels(ax.get_yticklabels(), rotation=0) # Keep y-axis labels horizontal

# 6. Tight layout for a professional look
plt.tight_layout()

return ax

```

Text Preprocessing

```

In [12]: import spacy
# Load spaCy pretraining model
nlp = spacy.load("en_core_web_sm")

def preprocess_and_tokenize(data):
    # Remove HTML notes
    data = re.sub("<.*?>", "", data)

    # Remove URL
    data = re.sub(r'http\S+', '', data)

    # Remove hashtags and @names
    data = re.sub(r"#[\d\w\.]+", '', data)
    data = re.sub(r"@[\d\w\.]+", '', data)

    # Remove punctuations and non-ASCII numbers
    data = re.sub(r"(\W|\d)", " ", data)

    doc = nlp(data)

    processed_tokens = []

    for token in doc:
        # Remove punctuations, spaces, numbers and stop words
        if not token.is_punct and not token.is_space and not token.is_digit and
            len(token.text.strip()) > 1: # Extract strings that length shorter or
                processed_tokens.append(token.lemma_.lower())

```

```
return processed_tokens
```

Text Representation

```
In [13]: # TFIDF, unigrams and bigrams
vect = TfidfVectorizer(tokenizer=preprocess_and_tokenize,
                       sublinear_tf=True, norm='l2',      # L2 normalization
                       ngram_range=(1, 2),              # Unigram and Bigram
                       min_df=2,                        # Ignore terms that appear less than 2 times
                       max_df=0.95,                     # Ignore terms that appear in more than 95% of documents
                       stop_words='english')

# fit on our complete corpus
vect.fit_transform(data.Text)

# transform testing and training datasets to vectors
X_train_vect = vect.transform(X_train)
X_test_vect = vect.transform(X_test)
```

Classifiers

Naive Bayes

```
In [14]: nb = MultinomialNB()

nb.fit(X_train_vect, y_train)

ynb_pred = nb.predict(X_test_vect)

nb_accuracy = accuracy_score(y_test, ynb_pred) * 100
nb_f1_score = f1_score(y_test, ynb_pred, average='micro') * 100

print("\nAccuracy (Naive Bayes): {:.2f}%".format(nb_accuracy))
print("\nF1 Score (Naive Bayes): {:.2f}%".format(nb_f1_score))
print("\nNaive Bayes Confusion Matrix:\n", confusion_matrix(y_test, ynb_pred))

# Plot normalized confusion matrix
plot_confusion_matrix(y_test, ynb_pred, classes=class_names, normalize=True, title='Naive Bayes Confusion Matrix')
plt.show()
```

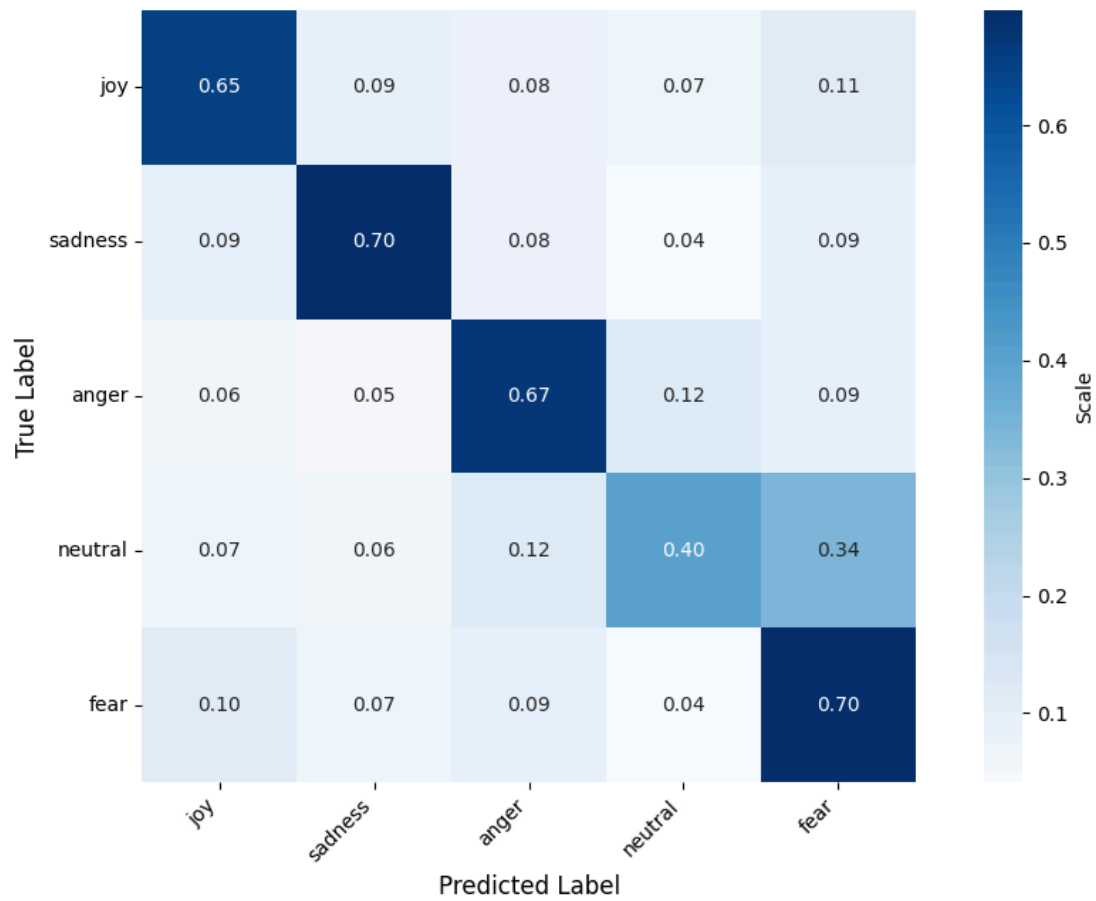
Accuracy (Naive Bayes): 62.81%

F1 Score (Naive Bayes): 62.81

Naive Bayes Confusion Matrix:

```
[[453  59  57  47  77]
 [ 62 474  55  28  60]
 [ 40  36 476  88  67]
 [ 46  39  78 258 217]
 [ 70  49  58  29 470]]
```

Normalized confusion matrix (Naive Bayes)



Random Forest

```
In [15]: rf = RandomForestClassifier(n_estimators=50)
rf.fit(X_train_vect, y_train)

yrf_pred = rf.predict(X_test_vect)

rf_accuracy = accuracy_score(y_test, yrf_pred) * 100
rf_f1_score = f1_score(y_test, yrf_pred, average='micro') * 100

print("\nAccuracy (Random Forest): {:.2f}%".format(rf_accuracy))
print("\nF1 Score (Random Forest): {:.2f}%".format(rf_f1_score))
print("\nRandom Forest Confusion Matrix:\n", confusion_matrix(y_test, yrf_pred))

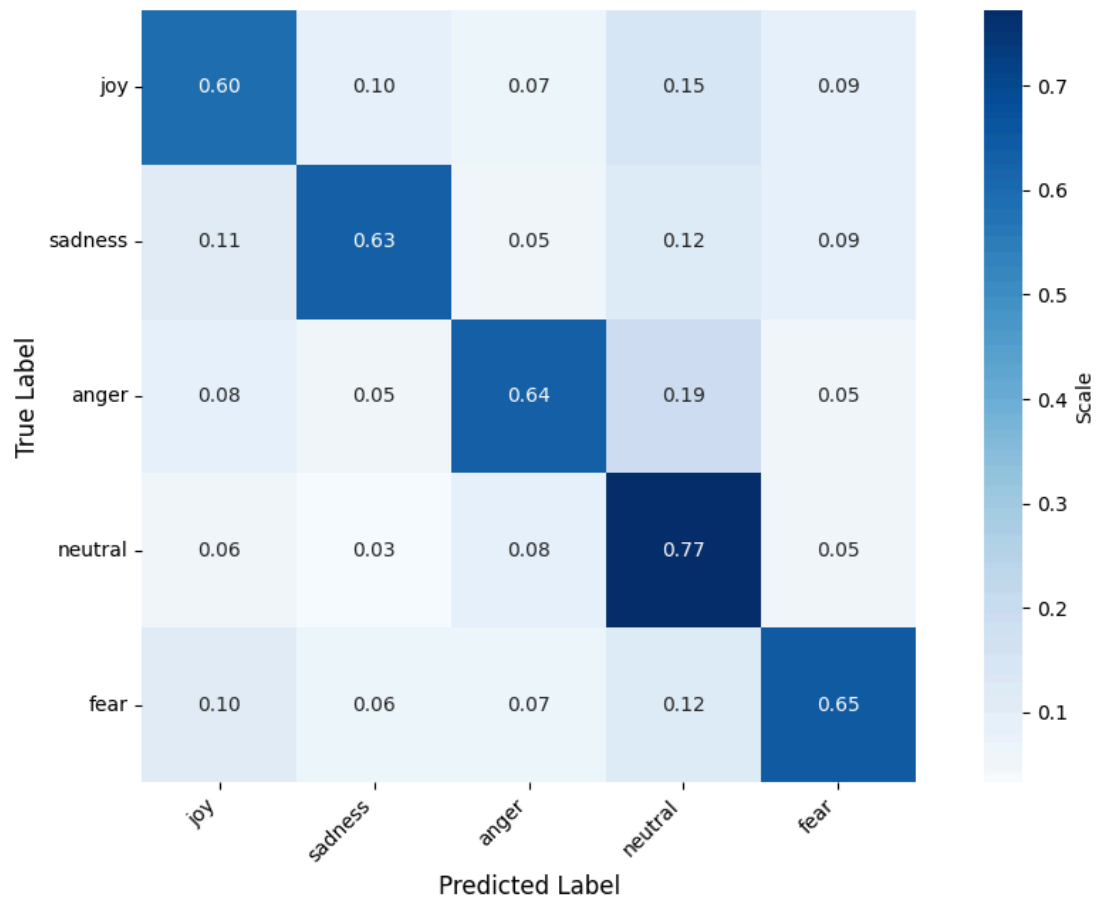
plot_confusion_matrix(y_test, yrf_pred, classes=class_names, normalize=True, title='Random Forest Confusion Matrix')
plt.show()
```

Accuracy (Random Forest): 65.55%

F1 Score (Random Forest): 65.55

Random Forest Confusion Matrix:

```
[[413  66  48 103  63]
 [ 73 430  33  83  60]
 [ 56  32 449 133  37]
 [ 38  21  54 493  32]
 [ 70  42  45  80 439]]
```

Normalized confusion matrix (Random Forest)

Logistic Regression

```
In [16]: log = LogisticRegression(solver='lbfgs', max_iter=200)
log.fit(X_train_vect, y_train)

ylog_pred = log.predict(X_test_vect)

log_accuracy = accuracy_score(y_test, ylog_pred) * 100
log_f1_score = f1_score(y_test, ylog_pred, average='micro') * 100

print("\nAccuracy (Logistic Regression): {:.2f}%".format(log_accuracy))
print("\nF1 Score (Logistic Regression): {:.2f}%".format(log_f1_score))
print("\nLogistic Regression Confusion Matrix:\n", confusion_matrix(y_test, ylog_pred))

plot_confusion_matrix(y_test, ylog_pred, classes=class_names, normalize=True, title='Logistic Regression Confusion Matrix')
plt.show()
```

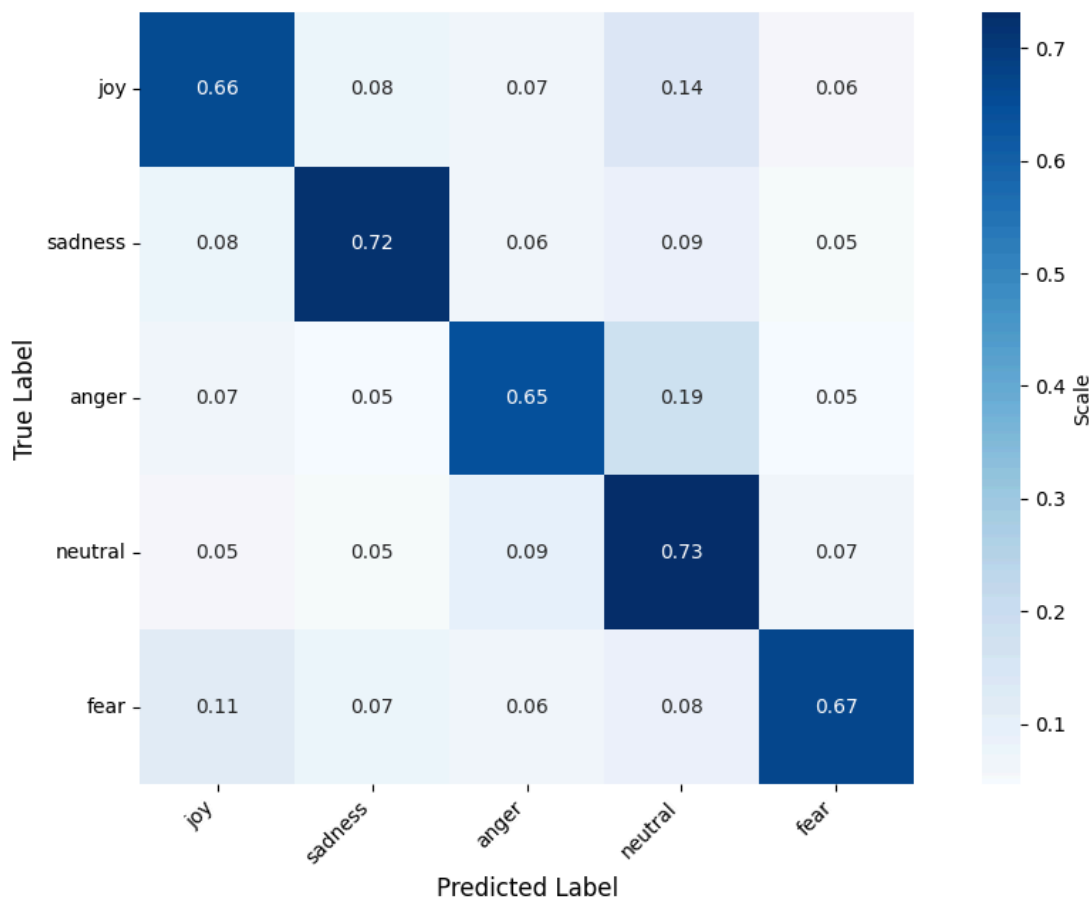
Accuracy (Logistic Regression): 68.41%

F1 Score (Logistic Regression): 68.41

Logistic Regression Confusion Matrix:

```
[[456 56 47 95 39]
 [ 55 489 39 60 36]
 [ 50 35 457 132 33]
 [ 35 34 59 467 43]
 [ 75 50 43 56 452]]
```

Normalized confusion matrix (Logistic Regression)



Linear Support Vector Classifier

```
In [17]: svc = SVC(tol=1e-05,
                  C=1.0,          # Regularization parameter
                  kernel='linear',
                  probability=True)

svc.fit(X_train_vect, y_train)

ysvm_pred = svc.predict(X_test_vect)

svc_accuracy = accuracy_score(y_test, ysvm_pred) * 100
svc_f1_score = f1_score(y_test, ysvm_pred, average='micro') * 100

print("\nAccuracy (Linear SVC): {:.2f}%".format(svc_accuracy))
print("\nF1 Score (Linear SVC): {:.2f}%".format(svc_f1_score))
print("\nLinear SVC Confusion Matrix:\n", confusion_matrix(y_test, ysvm_pred))

plot_confusion_matrix(y_test, ysvm_pred, classes=class_names, normalize=True, title='Confusion Matrix')
plt.show()
```

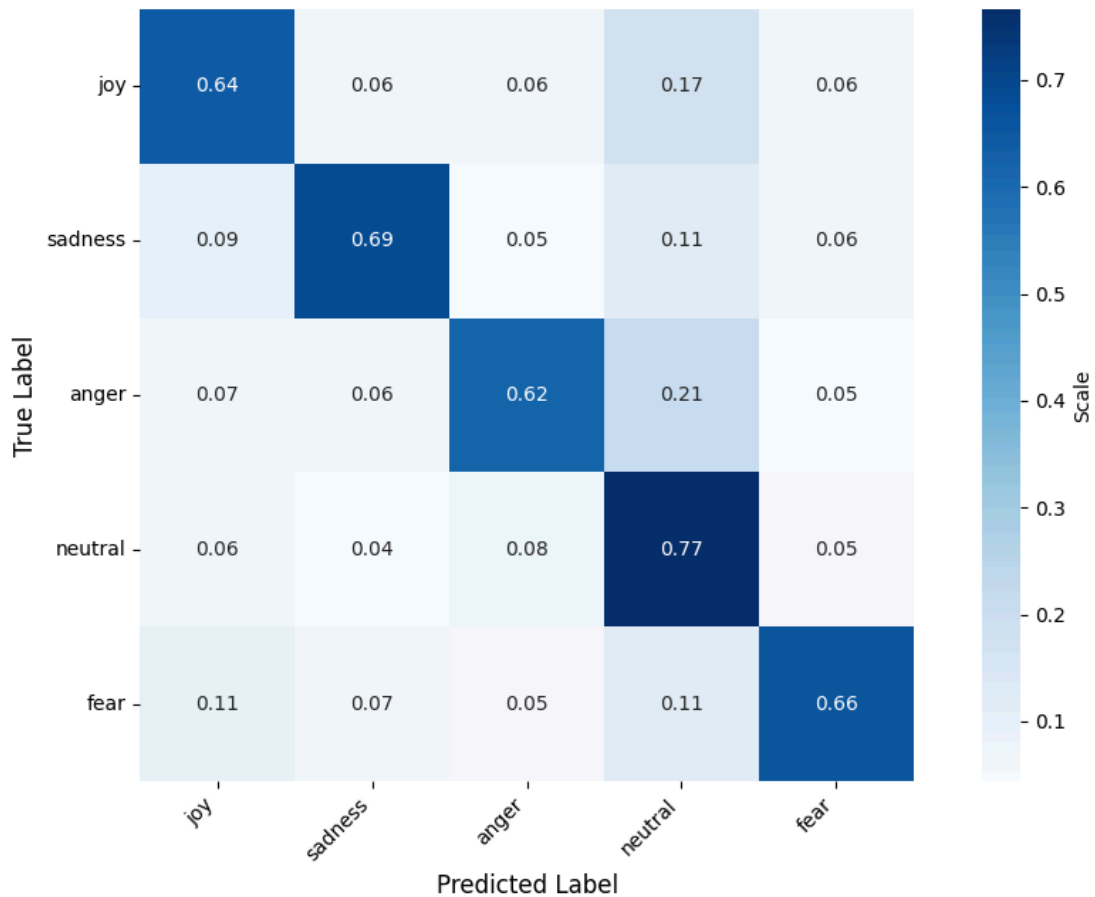
Accuracy (Linear SVC): 67.40%

F1 Score (Linear SVC): 67.40

Linear SVC Confusion Matrix:

```
[[442  45  45 118  43]
 [ 61 467  33  78  40]
 [ 46  43 440 146  32]
 [ 38  28  48 489  35]
 [ 73  44  36  74 449]]
```

Normalized confusion matrix (Linear SVC)



F1 Score Summary

```
In [18]: data = {
          'Approach': ['Naive Bayes', 'Random Forrest', 'Logistic Regression', 'SVM'],
          'F1-Score': [nb_f1_score, rf_f1_score, log_f1_score, svc_f1_score]
        }
df = pd.DataFrame(data)

df
```

Out[18]:

	Approach	F1-Score
0	Naive Bayes	62.805777
1	Random Forrest	65.546714
2	Logistic Regression	68.405541
3	SVM	67.403478

TF-IDF and SVM Model

```
In [19]: def create_emotion_classifier_pipeline(classifier):
        """
        Creates a complete pipeline for emotion classification.
        The pipeline handles both feature extraction and prediction.
        """
        emotion_pipeline = Pipeline([
            ('tfidf_vectorizer', vect),
            ('classifier', classifier)
        ])
        return emotion_pipeline

def save_emotion_model(pipeline, filepath):
    """Saves the trained emotion classification pipeline."""
    with open(filepath, 'wb') as model_file:
        pickle.dump(pipeline, model_file)

models = {'naive_bayes': nb,
          'random_forest': rf,
          'logistic_regression': log,
          'svm': svc}

for model_name, model in models.items():
    classifier_pipeline = create_emotion_classifier_pipeline(model)
    classifier_pipeline.fit(X_train, y_train)

    save_path = f'models/tfidf_{model_name}.sav'
    save_emotion_model(classifier_pipeline, save_path)
    print(f"Saved {model_name} model to {save_path}")

def predict_emotion(text_message, model_path):
    """Loads the model and predicts emotion for a given text."""
    with open(model_path, 'rb') as model_file:
        loaded_model = pickle.load(model_file)

    prediction = loaded_model.predict([text_message])
    probabilities = loaded_model.predict_proba([text_message])

    # print("训练时的类别顺序:", y_train.unique())
    # print("模型学习的类别顺序:", loaded_model.classes_)

    return prediction[0], probabilities[0]
```

Saved naive_bayes model to models/tfidf_naive_bayes.sav

Saved random_forest model to models/tfidf_random_forest.sav

Saved logistic_regression model to models/tfidf_logistic_regression.sav

Saved svm model to models/tfidf_svm.sav

```
In [20]: message = 'delivery was hour late and my pizza is cold!'
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}\n")

message = 'what do you mean'
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}\n")

message = 'too young too simple'
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}\n")

message = 'I cant decide what to do now, he is a freak'
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}\n")

message = 'it is really awful'
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_svm.sav')
print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}\n")
```

Message: 'delivery was hour late and my pizza is cold!'
Predicted emotion: anger
Confidence scores: [0.41201493 0.24856194 0.09456337 0.15685693 0.08800282]

Message: 'what do you mean'
Predicted emotion: neutral
Confidence scores: [0.08738695 0.14412758 0.11041116 0.54220025 0.11587406]

Message: 'too young too simple'
Predicted emotion: neutral
Confidence scores: [0.16072899 0.15143413 0.2253756 0.32647853 0.13598275]

Message: 'I cant decide what to do now, he is a freak'
Predicted emotion: fear
Confidence scores: [0.08721324 0.43274965 0.07698833 0.19523637 0.20781241]

Message: 'it is really awful'
Predicted emotion: sadness
Confidence scores: [0.08106924 0.04467994 0.09076149 0.29293278 0.49055655]

Emotion Classification in Texts using Transformer-based BERT Model

This implementation classifies text into five emotion categories (joy, sadness, anger, fear, neutral) using the BERT transformer architecture with transfer learning. The approach leverages pre-trained BERT embeddings fine-tuned for emotion detection, capturing deep contextual relationships and semantic nuances in text.

The pipeline covers text preprocessing compatible with BERT tokenization, model fine-tuning with learning rate scheduling, and comprehensive evaluation using accuracy, F1 scores, and confusion matrices to assess model performance on emotion classification tasks.

```
In [2]: # Importings

import pandas as pd
import numpy as np

# NumPy 2.0
if not hasattr(np, 'Inf'):
    np.Inf = np.inf
if not hasattr(np, 'NaN'):
    np.NaN = np.nan

import os
os.environ['TF_USE_LEGACY_KERAS'] = '1'

import ktrain
from ktrain import text

import warnings
warnings.filterwarnings('ignore')
```

Configuration and data loading

```
In [4]: class_names = ['joy', 'sadness', 'fear', 'anger', 'neutral']
encoding = {
    'joy': 0,
    'sadness': 1,
    'fear': 2,
    'anger': 3,
    'neutral': 4
}

num_labels = len(class_names)

MODEL_NAME = 'bert'
MAX_LEN = 350

In [ ]: data_train = pd.read_csv('data/data_train.csv', encoding='utf-8')
data_test = pd.read_csv('data/data_test.csv', encoding='utf-8')
```

```

X_train = data_train.Text.tolist()
y_train_int = [encoding[x] for x in data_train.Emotion.tolist()]

X_test = data_test.Text.tolist()
y_test_int = [encoding[x] for x in data_test.Emotion.tolist()]

data = pd.concat([data_train, data_test], ignore_index=True)

print('size of training set: %s' % (len(data_train['Text'])))
print('size of validation set: %s' % (len(data_test['Text'])))
print(data.Emotion.value_counts())

data.head(10)

```

```

size of training set: 7934
size of validation set: 3393
Emotion
joy      2326
sadness  2317
anger    2259
neutral  2254
fear     2171
Name: count, dtype: int64

```

Out[]:

	Emotion	Text
0	neutral	There are tons of other paintings that I thin...
1	sadness	Yet the dog had grown old and less capable , a...
2	fear	When I get into the tube or the train without ...
3	fear	This last may be a source of considerable disq...
4	anger	She disliked the intimacy he showed towards so...
5	sadness	When my family heard that my Mother's cousin w...
6	joy	Finding out I am chosen to collect norms for C...
7	anger	A spokesperson said : ` Glen is furious that t...
8	neutral	Yes .
9	sadness	When I see people with burns I feel sad, actua...

Tokenizers and data preprocessing

```

In [6]: (x_train, y_train), (x_test, y_test), preproc = text.texts_from_array(
        x_train=X_train, y_train=y_train_int,
        x_test=X_test, y_test=y_test_int,
        class_names=class_names,
        preprocess_mode=MODEL_NAME,
        maxlen=MAX_LEN,
        max_features=MAX_LEN*100)

```

```

preprocessing train...
language: en
done.

```

```

Is Multi-Label? False
preprocessing test...
language: en
done.
task: text classification

```

Model Initialization

```

In [7]: model = text.text_classifier(MODEL_NAME, train_data=(x_train, y_train), preproc=
Is Multi-Label? False
maxlen is 350
done.

```

Training parameters

```

In [6]: learner = ktrain.get_learner(model, train_data=(x_train, y_train),
val_data=(x_test, y_test),
batch_size=6)

```

Training and Evaluation

```

In [7]: learner.fit_onecycle(2e-5, 3)

begin training using onecycle policy with max lr of 2e-05...
Epoch 1/3
1323/1323 [=====] - 6253s 5s/step - loss: 0.9456 - accur
acy: 0.6352 - val_loss: 0.6144 - val_accuracy: 0.7819
Epoch 2/3
1323/1323 [=====] - 5836s 4s/step - loss: 0.4419 - accur
acy: 0.8493 - val_loss: 0.5422 - val_accuracy: 0.8126
Epoch 3/3
1323/1323 [=====] - 8836s 7s/step - loss: 0.1869 - accur
acy: 0.9420 - val_loss: 0.5592 - val_accuracy: 0.8243
Out[7]: <tf.keras.src.callbacks.History at 0x18ca9774590>

```

```

In [ ]: learner.validate(val_data=(x_test, y_test),
class_names=class_names)

```

```

107/107 [=====] - 1947s 18s/step
precision    recall  f1-score   support

   joy         0.85         0.85         0.85         707
  sadness       0.79         0.83         0.81         676
    fear       0.87         0.84         0.85         679
   anger       0.81         0.77         0.79         693
  neutral       0.80         0.83         0.81         638

 accuracy          0.82         0.82         0.82         3393
  macro avg       0.82         0.82         0.82         3393
 weighted avg     0.83         0.82         0.82         3393

```

```
Out[ ]: array([[604, 18, 15, 15, 55],
               [ 16, 561, 28, 50, 21],
               [ 18, 39, 567, 36, 19],
               [ 22, 65, 30, 537, 39],
               [ 48, 25, 9, 28, 528]])
```

```
In [8]: def f1_from_cm(confusion_matrix):
        n_classes = confusion_matrix.shape[0]
        f1_scores = []

        for i in range(n_classes):
            tp = confusion_matrix[i, i]                # True Positive
            fp = np.sum(confusion_matrix[i, :]) - tp   # False Positive
            fn = np.sum(confusion_matrix[:, i]) - tp   # False Negative

            precision = tp / (tp + fp) if (tp + fp) > 0 else 0
            recall = tp / (tp + fn) if (tp + fn) > 0 else 0

            f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

            f1_scores.append(f1)

            print(f"Class {i}: Precision={precision:.4f}, Recall={recall:.4f}, F1={f1:.4f}")

        return f1_scores

cm = np.array([[604, 18, 15, 15, 55],
               [ 16, 561, 28, 50, 21],
               [ 18, 39, 567, 36, 19],
               [ 22, 65, 30, 537, 39],
               [ 48, 25, 9, 28, 528]])
f1_scores = f1_from_cm(cm)

macro_f1 = np.mean(f1_scores) * 100
print(f"BERT average F1 score: {macro_f1:.2f}%")
```

```
Class 0: Precision=0.8543, Recall=0.8531, F1=0.8537
Class 1: Precision=0.8299, Recall=0.7924, F1=0.8107
Class 2: Precision=0.8351, Recall=0.8737, F1=0.8539
Class 3: Precision=0.7749, Recall=0.8063, F1=0.7903
Class 4: Precision=0.8276, Recall=0.7976, F1=0.8123
BERT average F1 score: 82.42%
```

```
In [ ]: predictor = ktrain.get_predictor(learner.model, preproc)
        predictor.get_classes()
        predictor.save('models/bert_model')
```

```
In [ ]: import time

        message = 'I just broke up with my boyfriend'

        start_time = time.time()
        prediction = predictor.predict(message)

        print('predicted: {} ({:.2f}s)'.format(prediction, (time.time() - start_time)))
```

```
predicted: sadness (0.26)
```

Model Testing

```
In [7]: import pandas as pd
from tabulate import tabulate
import re

import os
os.environ['TF_USE_LEGACY_KERAS'] = '1'

import ktrain

import pickle

import warnings
warnings.filterwarnings('ignore')
```

```
In [8]: import spacy
nlp = spacy.load("en_core_web_sm")

def preprocess_and_tokenize(data):
    data = re.sub("<.*?>", "", data)
    data = re.sub(r'http\S+', '', data)
    data = re.sub(r"#[\d\w\.]+", '', data)
    data = re.sub(r"@[\d\w\.]+", '', data)
    data = re.sub(r"(\W|\d)", " ", data)

    doc = nlp(data)

    processed_tokens = []

    for token in doc:
        if not token.is_punct and not token.is_space and \
            not token.is_digit and not token.is_stop and \
            len(token.text.strip()) > 1:
            processed_tokens.append(token.lemma_.lower())

    return processed_tokens
```

```
In [9]: def predict_emotion(text_message, model_path):
    """Loads the model and predicts emotion for a given text."""
    if model_path == 'models/bert_model':
        loaded_model = ktrain.load_predictor(model_path)
    else:
        with open(model_path, 'rb') as model_file:
            loaded_model = pickle.load(model_file)

    prediction = loaded_model.predict([text_message])
    probabilities = loaded_model.predict_proba([text_message])

    return prediction[0], probabilities[0]
```

Traditional ML Models

```
In [10]: # Choose model with best preformance, which is Logistic Regression

logistic_results = []

message = 'he is a freak'

# You can also change to different model paths if you want to test,
# according to the saved models in 'models' folder.
emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
max_confidence_score = max(confidence_scores)*100

logistic_results.append({'message': message, 'predicted_emotion': emotion, 'max_

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")

message = 'I cant decide what to do now, he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/tfidf_logistic_reg')
max_confidence_score = max(confidence_scores)*100

logistic_results.append({'message': message, 'predicted_emotion': emotion, 'max_

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")
```

```
Message: 'he is a freak'
Predicted emotion: fear
Confidence scores: [0.0853835  0.51886327 0.08929051 0.22434702 0.0821157 ]
Max confidence score (prediction): 51.89%
```

```
Message: 'I cant decide what to do now, he is a freak'
Predicted emotion: fear
Confidence scores: [0.08721324 0.43274965 0.07698833 0.19523637 0.20781241]
Max confidence score (prediction): 43.27%
```

Bert

```
In [11]: # BERT model that have been trained and saved already

bert_results = []

message = 'he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/bert_model')
max_confidence_score = max(confidence_scores)*100

bert_results.append({'message': message, 'predicted_emotion': emotion, 'max_conf

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")
```

```

message = 'I cant decide what to do now, he is a freak'

emotion, confidence_scores = predict_emotion(message, 'models/bert_model')
max_confidence_score = max(confidence_scores)*100

bert_results.append({'message': message, 'predicted_emotion': emotion, 'max_conf

print(f"Message: '{message}'")
print(f"Predicted emotion: {emotion}")
print(f"Confidence scores: {confidence_scores}")
print(f"Max confidence score (prediction): {max_confidence_score:.2f}%\n")

```

```

Message: 'he is a freak'
Predicted emotion: neutral
Confidence scores: [0.00208904 0.01453433 0.06090649 0.01062401 0.91184616]
Max confidence score (prediction): 91.18%

Message: 'I cant decide what to do now, he is a freak'
Predicted emotion: fear
Confidence scores: [4.7199646e-05 2.4234962e-03 9.9295467e-01 1.0142121e-03 3.560
4218e-03]
Max confidence score (prediction): 99.30%

```

Short Conclusion

Based on the comparative analysis, the **BERT transformer model** demonstrates significantly superior performance over **traditional Logistic Regression** for emotion classification.

BERT shows much higher prediction confidence (91.18-99.30% vs. 43.27-51.89%) and better contextual understanding.

For "he is a freak," Logistic Regression incorrectly predicts "**fear**" with low confidence, while BERT correctly identifies "**neutral**" context with high certainty. Both models agree on the extended sentence, but BERT maintains exceptional confidence.

The substantial confidence gap and BERT's nuanced semantic processing highlight transformer models' advantage in capturing emotional subtleties and contextual meaning beyond traditional bag-of-words approaches.

```

In [15]: summary_stats = []

for i in range(len(logistic_results)):
    log_max = logistic_results[i]['max_confidence']
    bert_max = bert_results[i]['max_confidence']
    confidence_gap = bert_max - log_max

    summary_stats.append({
        'Message': logistic_results[i]['message'][:18] + "...",
        'LR_Pred': logistic_results[i]['predicted_emotion'],
        'BERT_Pred': bert_results[i]['predicted_emotion'],
        'LR_Confi': f"{log_max:.2f}%",
        'BERT_Confi': f"{bert_max:.2f}%",
        'Confi_Diff': f"{confidence_gap:+.2f}%",
    })

```

```

        'Same_Pred': 'Yes' if logistic_results[i]['predicted_emotion'] == bert_r
    })

summary_df = pd.DataFrame(summary_stats)
print("")
print(tabulate(summary_df, headers='keys', tablefmt='grid', showindex=False))

print(f"\nAverage Confidence Scores:")
print(f"  Logistic Regression: {(logistic_results[0]['max_confidence'] + logisti
print(f"  BERT Transformer: {(bert_results[0]['max_confidence'] + bert_results[1
print(f"\nPerformance Improvement: {(bert_results[0]['max_confidence'] + bert_r

```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+
| Message          | LR_Pred   | BERT_Pred  | LR_Confi   | BERT_Confi | C
onfi_Diff   | Same_Pred |
+-----+-----+-----+-----+-----+-----+
+-----+-----+
| he is a freak    | fear      | neutral    | 51.89%     | 91.18%     | +
39.30%      | No        |
+-----+-----+-----+-----+-----+-----+
+-----+-----+
| I cant decide what... | fear      | fear        | 43.27%     | 99.30%     | +
56.02%      | Yes       |
+-----+-----+-----+-----+-----+-----+
+-----+-----+

```

Average Confidence Scores:

Logistic Regression: 47.58%

BERT Transformer: 95.24%

Performance Improvement: 47.66%

B References

References

- [1] Murthy, A. R., & Kumar, K. A. (2021, March). A review of different approaches for detecting emotion from text. In *IOP Conference Series: Materials Science and Engineering* (Vol. 1110, No. 1, p. 012009). IOP Publishing.
- [2] Acheampong, F. A., Wenyu, C., & Nunoo-Mensah, H. (2020). Text-based emotion detection: Advances, challenges, and opportunities. *Engineering Reports*, 2(7), e12189.
- [3] Aman, S., & Szpakowicz, S. (2007, September). Identifying expressions of emotion in text. In *International conference on text, speech and dialogue* (pp. 196-205). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [4] Alm, C. O., Roth, D., & Sproat, R. (2005, October). Emotions from text: machine learning for text-based emotion prediction. In *Proceedings of human language technology conference and conference on empirical methods in natural language processing* (pp. 579-586).